COURSE OUTCOME 3 (CO3)

Smart University Timetable Generator

Using Constraint Satisfaction Problem (CSP)

Backtracking | AC-3 | MRV | Degree | LCV

Python | Flask | SQLite | HTML5 | CSS3 | JavaScript

Project Report -- CO3 Implementation

Generated: 03 June 2026, 10:26 AM

Academic Year 2024-25

Table of Contents

1. Project Overview

Smart University Timetable Generator is a full-stack web application that automatically creates conflict-free university class schedules using Artificial Intelligence. It implements Course Outcome 3 (CO3) -- Constraint Satisfaction Problems -- using five distinct algorithms: Backtracking Search, AC-3 Constraint Propagation, MRV Heuristic, Degree Heuristic, and LCV (Least Constraining Value).

Technology Stack

Backend: Python 3, Flask web framework

Database: SQLite (built-in sqlite3 module -- no external DB server needed)

Frontend: HTML5, CSS3 (Dark/Light theme), JavaScript ES6+

Libraries: fpdf2 (PDF export), openpyxl (Excel export), Werkzeug (password hashing)

2. Complete File Structure

Project Directory Tree

```
CFAI B project/
  app.py                  <- Flask server + ALL routes (650+ lines)
  database.db             <- SQLite database (auto-created on startup)
  requirements.txt        <- Python package list
  models/
    user.py               <- User authentication + password hashing
    timetable.py          <- All DB operations + analytics logic
  algorithms/             <- CO3 CSP Implementation
    csp.py                <- Core CSP class + AC-3 propagation
    backtracking.py       <- Backtracking search engine + entry point
    mrv.py                <- MRV heuristic (variable ordering)
    degree.py             <- Degree heuristic (tie-breaking)
    lcv.py                <- LCV heuristic (value ordering)
  templates/
    base.html             <- Shared layout: sidebar, topbar, theme toggle
    landing.html          <- Public home page with hero section
    login.html            <- Glassmorphism login form
    register.html         <- Role-selector registration form
    teacher/
      dashboard.html      <- Stats cards, charts, activity log
      timetable.html      <- CSP generate + grid view + conflicts
      subjects.html       <- CRUD for subjects (add/edit/delete)
      teachers.html       <- CRUD for teachers
      classrooms.html     <- CRUD for classrooms
    student/
      dashboard.html      <- Today's classes + weekly mini-grid
      timetable.html      <- Search + filter + full timetable grid
  static/
    css/style.css         <- 1000+ lines: dark/light theme, all components
    js/app.js             <- Client JS: search, charts, modals, fetch
```

3. Layer-by-Layer Architecture

The application follows a classic 3-layer architecture:

Layer	Technology	Responsibility
Presentation	HTML5 + CSS3 + JS	UI rendering, forms, real-time search, charts, dark mode
Application	Python + Flask	Routing, session auth, business logic, CSP solver calls
Data	SQLite	Persistent storage of users, subjects, teachers, timetable

AI Engine	algorithms/ package	CSP modeling, AC-3, MRV, Degree, LCV, Backtracking search

Data flow: Browser sends request -> Flask routes handle it -> CSP algorithms solve the problem -> SQLite stores/retrieves data -> Flask renders HTML -> Browser displays result.

4. Backend -- app.py

app.py is the central hub. It initializes Flask, creates the SQLite database, and defines every URL route. It uses function decorators to enforce access control.

Database Initialization (init_db)

Called once on startup. Creates 7 tables and seeds 30 default time slots (Mon-Fri, 6 slots/day, 9:00-16:00).

Route Summary

Route	Method	Purpose	Access
/	GET	Public landing page	Public
/login	GET/POST	Login form + session creation	Public
/register	GET/POST	Registration + password hash	Public
/logout	GET	Clear session	Logged in
/teacher/dashboard	GET	Stats, charts, activity log	Teacher
/teacher/subjects	GET/POST	Add/Edit/Delete subjects	Teacher
/teacher/teachers	GET/POST	Add/Edit/Delete teachers	Teacher
/teacher/classrooms	GET/POST	Add/Edit/Delete rooms	Teacher
/teacher/timetable	GET	View grid + conflict display	Teacher
/teacher/timetable/generate	POST	Run CSP solver, save result	Teacher
/teacher/export/pdf	GET	Download PDF via fpdf2	Teacher
/teacher/export/excel	GET	Download Excel via openpyxl	Teacher
/student/dashboard	GET	Today's classes + weekly view	Student
/student/timetable	GET	Full timetable + search/filter	Student
/api/conflicts	GET	JSON: list of detected conflicts	Logged in

Access Control

Decorator Logic (app.py)

```
@login_required   -> Checks if session["user_id"] exists
                     If not -> redirect to /login

@teacher_required -> Checks session["role"] == "teacher"
                     Students who try teacher URLs get redirected

log_activity()    -> Inserts a row into activity_log for every
                     significant teacher action (generate, add, delete...)
```

5. Database Design -- 7 Tables

Table: users

Column	Type	Description
id	INTEGER PK	Auto-increment primary key
fullname	TEXT	Full name e.g. "Dr. Ramesh Sharma"
email	TEXT UNIQUE	Login identifier -- must be unique
password	TEXT	Werkzeug hash (PBKDF2-SHA256) -- never plain text
role	TEXT	"teacher" or "student"
created_at	TEXT	Registration timestamp (ISO format)

Table: subjects

Column	Type	Description
id	INTEGER PK	Primary key
subject_name	TEXT	e.g. "Data Structures and Algorithms"
credits	INTEGER	Academic credits (default: 3)
hours_per_week	INTEGER	Teaching hours per week (default: 3)

Table: teachers

Column	Type	Description
id	INTEGER PK	Primary key
teacher_name	TEXT	e.g. "Dr. Patel"
specialization	TEXT	e.g. "Algorithms and AI"

Table: classrooms

Column	Type	Description
id	INTEGER PK	Primary key
room_number	TEXT	e.g. "CS-101", "Lab-A"
capacity	INTEGER	Seating capacity (default: 60)

Table: timeslots (auto-seeded with 30 rows)

Column	Type	Description
id	INTEGER PK	Primary key
day	TEXT	"Monday", "Tuesday", ... "Friday"
start_time	TEXT	"09:00", "10:00", ... "15:00"
end_time	TEXT	"10:00", "11:00", ... "16:00"

Table: timetable (THE RESULT TABLE)

Column	Type	Description
id	INTEGER PK	Primary key
subject_id	INTEGER FK	References subjects.id
teacher_id	INTEGER FK	References teachers.id
classroom_id	INTEGER FK	References classrooms.id
timeslot_id	INTEGER FK	References timeslots.id

The timetable table stores one row per scheduled class. When you click "Generate", ALL old rows are DELETED and replaced with the new CSP solution. The CSP solver guarantees no two rows share the same (teacher_id + timeslot_id) OR (classroom_id + timeslot_id) -- meaning zero conflicts.

Table: activity_log

Column	Type	Description
id	INTEGER PK	Primary key
user_id	INTEGER FK	Which user performed the action
action	TEXT	"GENERATE_TIMETABLE", "ADD_SUBJECT", "LOGIN", etc.
detail	TEXT	e.g. "Solved in 312ms, 0 backtracks"
timestamp	TEXT	Date and time of the action

6. CO3 Algorithms -- Deep Dive

6.1 CSP Modeling (algorithms/backtracking.py)

The timetable problem is modeled as a Constraint Satisfaction Problem. Every CSP has three components:

CSP Formulation

```
VARIABLES  = Each subject that needs to be scheduled
             Example: {Math, Physics, AI, Chemistry, OS}

DOMAINS    = All valid (teacher, classroom, timeslot) combinations per subject
             Example: Math can be assigned:
               (Dr. Sharma, Room CS-101, Monday 09:00)
               (Dr. Sharma, Room CS-101, Monday 10:00)
               (Dr. Patel,  Room CS-102, Tuesday 09:00)
               ... (hundreds of combinations)

CONSTRAINTS = Rules for a valid assignment:
  Hard Constraint 1: Two subjects at the SAME timeslot need DIFFERENT teachers
  Hard Constraint 2: Two subjects at the SAME timeslot need DIFFERENT classrooms
```

6.2 AC-3 -- Arc Consistency Propagation (algorithms/csp.py)

AC-3 is run BEFORE the search begins. It removes impossible values from domains so backtracking has less work. If any domain empties, there is no solution.

AC-3 Algorithm (csp.py)

```
ALGORITHM AC-3:
  queue = all (Xi, Xj) constraint pairs
  while queue is not empty:
    (Xi, Xj) = queue.pop()
    if REVISE(Xi, Xj):            <- Did any values get removed from Xi?
      if domain[Xi] is empty:
        return FAILURE             <- No solution possible at all
      for each Xk in neighbors[Xi]:
        queue.add((Xk, Xi))        <- Propagate the shrinkage
  return SUCCESS

REVISE(Xi, Xj):
  for each value vi in domain[Xi]:
    if NO value vj in domain[Xj] satisfies constraint(vi, vj):
      remove vi from domain[Xi]    <- Prune impossible value!
  return True if anything was removed

MAC (Maintaining Arc Consistency):
  After each assignment during backtracking, run focused AC-3
  on the assigned variable's neighbors only.
  Catches dead-ends immediately without waiting for failure.
```

6.3 MRV -- Minimum Remaining Values (algorithms/mrv.py)

MRV answers: "Which subject should we assign next?" Pick the one with the FEWEST valid options left. This is the "most constrained variable" heuristic.

MRV Heuristic (mrv.py)

```
ALGORITHM MRV:
  unassigned = [subjects not yet assigned]
  min_size = minimum(len(domain[v]) for v in unassigned)
  candidates = [v for v in unassigned if len(domain[v]) == min_size]
  return candidates  <- may be multiple (tie) -> Degree breaks ties

Example:
  Math:      120 valid slots remaining
  Physics:     3 valid slots remaining  <- MRV picks this first!
  Chemistry:  45 valid slots remaining

Why? If Physics only has 3 options, delaying it risks finding
     later that all 3 are gone -> lots of backtracking wasted.
     Assign the hardest variable FIRST to detect failures early.
```

6.4 Degree Heuristic (algorithms/degree.py)

The Degree heuristic is a TIE-BREAKER for MRV. When multiple subjects share the same minimum domain size, pick the one with the MOST constraints on other still-unassigned subjects.

Degree Heuristic (degree.py)

```
ALGORITHM DEGREE:
  for each candidate (already MRV-tied):
    degree = count(constraints with still-unassigned neighbors)
  return candidate with HIGHEST degree

Example (both have 3 slots remaining):
  Physics: constrained with Math, Chemistry, AI    -> degree = 3
  History: constrained with English only           -> degree = 1

  Pick Physics! Its assignment affects more other subjects,
  providing more information and reducing domains faster.

In code: select_variable() in degree.py calls mrv() first,
         then degree_heuristic() to break ties.
```

6.5 LCV -- Least Constraining Value (algorithms/lcv.py)

LCV answers: "We picked the subject -- in what ORDER do we try its values?" Sort values by how many options they ELIMINATE for neighbors. Try the least constraining value first.

LCV Heuristic (lcv.py)

```
ALGORITHM LCV:
  for each value v in domain[subject]:
    conflict_count(v) = number of neighbor values eliminated
                        if v is assigned to subject
  return sorted(domain_values, key=conflict_count)  # ascending

Example (assigning Math):
  Option A: (Dr. Sharma, Room CS-101, Mon 09:00)
    -> Removes 15 options from Physics, 8 from Chemistry = 23 total

  Option B: (Dr. Patel, Room CS-102, Wed 14:00)
    -> Removes 2 options from Physics, 1 from Chemistry = 3 total

  LCV picks Option B first! Leaves maximum flexibility for others.
  This maximizes the chance of finding a complete solution.
```

6.6 Backtracking Search + MAC (algorithms/backtracking.py)

The master algorithm. Orchestrates all four heuristics into one complete search:

Backtracking Pseudocode with MAC + MRV + LCV

```
FUNCTION generate_timetable(subjects, teachers, classrooms, timeslots):

  STEP 1 - Build CSP:
    variables = [subject.id for each subject]
    domains   = {subject_id: [(teacher_id, room_id, slot_id) for ALL combos]}
    constraints = pairwise between every 2 subjects:
      if same timeslot -> must differ in teacher AND room

  STEP 2 - AC-3 Preprocessing:
    domains = ac3(domains)
    if any domain empty: return "No solution found"

  STEP 3 - Backtracking:
    def backtrack(assignment, domains):
      if len(assignment) == len(variables): return assignment  # Done!

      var = select_variable(MRV + Degree)  # Hardest subject first
      values = lcv(var, domains[var], ...)  # Best values first

      for value in values:
        if is_consistent(var, value, assignment):
          assignment[var] = value
          new_domains = MAC(var, domains)  # Propagate via AC-3
          if new_domains is not None:       # Domain not wiped out?
            result = backtrack(assignment, new_domains)
            if result: return result         # SUCCESS!
          del assignment[var]               # Backtrack!
          metrics["backtracks"] += 1
      return None  # Try different value for parent

  STEP 4 - Convert solution:
    Map IDs -> full names, compute quality score, return assignments
```

7. Authentication System

The app uses Flask session-based authentication with Werkzeug password hashing. Passwords are stored as PBKDF2-SHA256 hashes with a random salt.

Auth Flow (app.py)

```
REGISTRATION:
  1. User submits: fullname, email, password, confirm_password, role
  2. Validate: fields present, passwords match, min 6 characters
  3. Check: email not already in database
  4. Hash: generate_password_hash("mypassword")
      -> "pbkdf2:sha256:600000$randomsalt$hexhash"
  5. Store hash in users table (plain text password NEVER stored)

LOGIN:
  1. User submits: email, password
  2. Fetch user row by email from DB
  3. Verify: check_password_hash(stored_hash, entered_password)
  4. If match: session["user_id"] = id, session["role"] = "teacher"/"student"
  5. Redirect to correct dashboard

LOGOUT:
  session.clear() -> destroys all session data
```

8. User Roles -- Teacher vs Student

Feature	Teacher	Student
Register / Login	YES	YES
Generate CSP Timetable	YES	NO
Manage Subjects / Teachers / Classrooms	YES	NO
Manually Add/Delete Timetable Entries	YES	NO
View Timetable Grid	YES	YES
Search and Filter Timetable	YES	YES
View Conflict Detection	YES	NO
View Analytics Charts	YES	NO
View Activity Audit Log	YES	NO
Export as PDF	YES	YES
Export as Excel	YES	NO
View Today's Classes on Dashboard	YES	YES

9. Frontend -- HTML, CSS, JavaScript

Template Inheritance (Jinja2)

All dashboard pages extend base.html using Jinja2 template inheritance. base.html provides the sidebar, topbar, flash messages, profile modal. Child pages only define their unique content inside {% block content %}.

Template Inheritance Structure

```
base.html structure:
  <aside class="sidebar">
    - Role-based nav links (teacher sees different items than student)
    - User avatar, name, role displayed at bottom
    - Logout link
  </aside>
  <div class="main-area">
    <div class="topbar">  <- Page title + dark mode toggle button
    Flash messages        <- Success/Error alerts from Flask
    {% block content %}   <- Child page inserts its own HTML here
  </div>
  Profile popup modal     <- Shown when clicking Profile nav item
```

style.css -- Design System (1000+ lines)

Uses CSS custom properties (variables) for theming. Two complete themes in one file:

CSS Theme Variables (style.css)

```
:root {                          /* LIGHT THEME (default) */
  --bg-body:        #f0f2ff;
  --bg-card:        #ffffff;
  --text-primary:   #1e1b4b;
  --accent:         #6366f1;     /* Indigo */
  --shadow-md:      0 4px 16px rgba(99,102,241,.12);
}

[data-theme="dark"] {            /* DARK THEME override */
  --bg-body:        #0a0a1a;
  --bg-card:        #11112b;
  --text-primary:   #e2e8f0;
  --border:         rgba(255,255,255,.08);
}

/* Switching theme: JS sets data-theme on <html> element */
/* All components automatically update -- no JS required! */
```

Key CSS components:

CSS Class	Purpose
.auth-card	Glassmorphism login/register card with blur + semi-transparency
.hero	Landing page hero with animated gradient floating orbs
.sidebar	Dark navigation sidebar with role-based menu items
.stat-card	Dashboard metric cards with colored icon boxes + hover lift
.tt-grid	CSS Grid timetable with dynamic day columns
.tt-entry .tt-c0..c9	10 color-coded class block styles (one per subject)
.conflict-banner	Red/Green strip with pulse animation for conflict status
.bar-chart	Animated horizontal bar charts for analytics data
.modal-overlay	Full-screen overlay + centered popup modal system
.btn-generate	Primary generate button with animated shimmer effect

app.js -- Client-Side JavaScript

Function	Purpose
themeToggle listener	Toggles data-theme attr, saves preference to localStorage
showToast(msg, type)	Shows floating notification auto-dismissed after 3.5 sec
openModal / closeModal	CSS class toggle to show/hide modal popups
fetch /timetable/generate	Calls CSP solver, animates 6 step progress, shows metrics
animateCounter(el, target)	Counts up from 0 to value with cubic easing animation
buildTimetableGrid(...)	Dynamically builds entire CSS Grid DOM from JSON data
getCourseColor(name)	Returns consistent color index (0-9) per subject name
initFilters()	Real-time search: dims non-matching timetable entries on input
renderBarChart(...)	Builds bar chart HTML with animated CSS width transitions
observeCharts()	Uses IntersectionObserver to animate bars when scrolled into view
initTableSearch()	Hides table rows not matching search text (instant)

10. Export System -- PDF and Excel

PDF Export (fpdf2)

When Export PDF is clicked, Flask fetches all timetable entries via a JOIN query, builds a formatted PDF in memory using fpdf2, and sends it directly as a download. No file is ever saved to disk.

PDF Export Code (app.py)

```
pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", "B", 16)
pdf.cell(0, 12, "Smart University Timetable", align="C")

# Colored header row
pdf.set_fill_color(79, 70, 229)
for label, width in columns:
    pdf.cell(width, 9, label, border=1, fill=True)

# Alternating row colors
pdf_bytes = pdf.output()           # Returns bytes, no file written
return send_file(io.BytesIO(pdf_bytes), as_attachment=True)
```

Excel Export (openpyxl)

Creates a styled workbook with indigo column headers, alternating row colors, and auto column widths. Also returned as in-memory bytes -- no file saved to disk.

11. Complete Generation Flow (End-to-End)

Complete Pipeline

```
STEP 1: Teacher clicks "Solve CSP & Generate Timetable" button

STEP 2: JavaScript:
  fetch("/teacher/timetable/generate", { method: "POST" })
  Shows animated 6-step progress overlay

STEP 3: Flask route (app.py -> generate_timetable):
  subjects   = SELECT * FROM subjects
  teachers   = SELECT * FROM teachers
  classrooms = SELECT * FROM classrooms
  timeslots  = SELECT * FROM timeslots
  result = algorithms.backtracking.generate_timetable(...)

STEP 4: backtracking.py -> generate_timetable():
  csp = build_timetable_csp(...)    <- Create variables + domains + constraints
  domains = AC-3 preprocessing      <- Prune impossible values
  solution = _backtrack({}, domains)
    Uses MRV + Degree to pick each subject variable
    Uses LCV to order values to try
    Uses MAC (AC-3) after each assignment
    Backtracks when stuck

STEP 5: Flask saves to DB:
  DELETE FROM timetable
  INSERT INTO timetable for each assignment in solution

STEP 6: Returns JSON: { success, assignments, metrics }

STEP 7: JavaScript:
  Animates metric counters (backtracks, solve time, checks...)
  Shows success toast notification
```

```
  Reloads page after 1.2 seconds

STEP 8: Teacher sees color-coded timetable grid with 0 conflicts
```

12. Algorithm Metrics Explained

Metric	What It Means	Good Value
Solve Time (ms)	Total time to find a complete solution	Under 1000ms
Backtracks	Times algorithm undid an assignment and tried another	0 = heuristics worked perfectly
Constraint Checks	Number of times a constraint function was evaluated	Lower = better pruning by AC-3
Variables	Number of subjects successfully scheduled	Should equal number of subjects
Constraints	Pairwise conflict rules: n*(n-1)/2	Grows quadratically with subjects
Quality Score (%)	How spread classes are across all 5 days	Higher = better daily distribution

Example Metrics After Generation

Solve Time: 312 ms

Backtracks: 0 (MRV + LCV + AC-3 found solution on first try -- optimal!)

Constraint Checks: 847 (each check tested one subject-pair constraint)

Variables: 8 (8 subjects scheduled)

Constraints: 28 (8 * 7 / 2 = 28 pairwise rules)

Quality Score: 80% (4 of 5 weekdays have at least one class)

13. How to Run the Project

Terminal Commands (Windows PowerShell / CMD)

```
# STEP 1: Open a terminal and go to the project folder
cd "CFAI B project"

# STEP 2: Install Python dependencies (only needed once)
pip install flask flask-login werkzeug fpdf2 openpyxl

# STEP 3: Start the Flask development server
python app.py

# Output you should see:
#  * Running on http://127.0.0.1:5000
#  * Debugger is active!

# STEP 4: Open your browser and visit:
http://localhost:5000

# STEP 5: Register as a Teacher
#  -> Go to: Manage > Subjects  -> Add 3-5 subjects
#  -> Go to: Manage > Teachers  -> Add 2-3 teachers
#  -> Go to: Manage > Classrooms -> Add 2 classrooms
#  -> Go to: Timetable -> Click "Solve CSP & Generate"
#  -> Watch the AI generate a conflict-free schedule!

# STEP 6: Register as a Student (separate browser tab)
#  -> View the generated timetable
#  -> Use search/filter, download PDF
```

Minimum Data Required Before Generating

At least 1 subject, 1 teacher, and 1 classroom must be added.

Time slots are auto-seeded (30 slots: Mon-Fri x 6 slots per day).

Tip: Start with 3-5 subjects, 3 teachers, 2 classrooms for best results.

If you get "No solution found", add more classrooms or teachers.

14. CO3 Topic Mapping Summary

This table maps every CO3 exam topic to its exact implementation in the codebase:

CO3 Topic	Algorithm	File	Function / Where
CSP Modeling	CSP class	csp.py	class CSP, add_variable(), add_constraint()
Variables	TimetableCSP	backtracking.py	subjects = variables in build_timetable_csp()
Domains	TimetableCSP	backtracking.py	all (teacher, room, slot) combos = domain
Constraints	Binary constraints	backtracking.py	make_constraint() -- same timeslot check
Backtracking	_backtrack()	backtracking.py	Recursive search with assignment undo
Arc Consistency	AC-3	csp.py	ac3() and _revise() methods
Constraint Propagation	MAC	csp.py	mac() -- focused AC-3 after each assign
MRV Heuristic	Min Remaining Values	mrv.py	mrv() function
Degree Heuristic	Constraint Degree	degree.py	degree_heuristic() and select_variable()
LCV Heuristic	Least Constraining	lcv.py	lcv() with conflict_count() helper
Solution Checking	Consistency test	csp.py	is_consistent() method
Forward Checking	Domain pruning	csp.py	MAC after every assignment
Soft Constraints	Quality score	backtracking.py	_evaluate_quality() -- day spread

Key Definitions for Exams

MRV = choose the VARIABLE with fewest remaining values next

Degree = break MRV ties by choosing variable with most constraints on others

LCV = for chosen variable, try VALUE that eliminates fewest options for neighbors

AC-3 = preprocessing: remove domain values with no support in neighbor domains

MAC = run AC-3 after every assignment to maintain arc consistency

Backtracking = try a value; if it leads to failure, undo and try another

Project Complete!

Smart University Timetable Generator successfully implements all CO3 -- Constraint Satisfaction Problem topics in a production-quality full-stack web application.

Backtracking Search	Core recursive search with assignment undo
AC-3 Propagation	Pre-prunes impossible domain values before search
MRV Heuristic	Most constrained variable assigned first
Degree Heuristic	Highest constraint count as tiebreaker
LCV Heuristic	Least constraining value tried first

Generated by UniSchedule | 03 June 2026